Lecture 6 — Advanced Strings: Suffix Array + LCP, Suffix Automaton

> National Championship Prep · Prerequisites: Year weeks 28–29 (hashing, prefix function, Z-function, tries, suffix array intro), Year week 24 (sparse tables), Regional 09 (strings for regionals), Nationals 02 & 04 (mergeable segment trees, small-to-large) for the stretch applications · Estimated time: 2 × 75 min lecture + 3 h practice

1. Overview

At regional level most string problems fall to hashing, KMP/Z, or Manacher. At the national level the problems change character: you are asked to **aggregate over all substrings** ("sum over every distinct substring of f(number of occurrences)"), to answer **many substring queries** ("how many times does s[l..r] occur?"), or to compare substrings of several strings at once. There are $\Theta(n^2)$ substrings, so the only hope is a structure that groups them into O(n) classes.

Two structures do this and you should be fluent in both:

- The **suffix array (SA)** with its **LCP array**. Suffixes are sorted; substrings with a given prefix form a contiguous block; the LCP array plus range-minimum queries gives the longest common prefix of any two suffixes in O(1). The implicit tree of "LCP intervals" is the suffix tree.
- The **suffix automaton (SAM)**: the minimal DFA recognising all substrings of s. Its states are classes of substrings with the same set of end positions; its suffix links form a tree. It is built online in amortized linear time.

This lecture gives both constructions with correctness and complexity arguments, then a catalogue of applications that appear repeatedly in national contests. (The World Finals track, Lecture 8, continues with eertrees, suffix trees, and Lyndon factorization.)

2. Suffix array by prefix doubling

2.1 Definition

For a string s of length n, `sa` is the permutation of `0..n-1` such that the suffixes `s[sa[0]..]`, `s[sa[1]..]`, … are in increasing lexicographic order. `rank[sa[i]] = i` is its inverse.

```
s = banana
i  sa[i]  suffix   lcp[i] = LCP(suffix sa[i], suffix sa[i+1])
```

```
0  5      a        1
1  3      ana      3
2  1      anana    0
3  0      banana   0
4  4      na       2
5  2      nana     -
```

The single most important observation: **all suffixes starting with a given string p occupy a contiguous range of sa**, because they are exactly the suffixes that are ≥ p and < (p followed by "infinity"). Every SA application is built on this.

2.2 Doubling

Append a sentinel `'\0'` smaller than every real character and consider **cyclic shifts** of `t = s + '\0'` (length N = n+1). Because the sentinel occurs once and is smallest, comparing two cyclic shifts is decided no later than the sentinel, so cyclic order equals suffix order.

Let `c_k[i]` be the equivalence class (an integer in sorted order) of the length-2^k cyclic substring starting at i. Then the length-2^{k+1} substring at i is the concatenation of the length-2^k substrings at i and i+2^k, so its order is determined by the **pair** `(c_k[i], c_k[i+2^k])`. Sorting the pairs gives `c_{k+1}`. After $\lceil \log_2 N \rceil$ rounds the substrings are longer than N, all classes are distinct, and the order is the suffix order.

Making a round O(n). Suppose `p` is the order of shifts by their length-2^k prefixes. Then the list `p[i] − 2^k (mod N)` is exactly the list of shifts ordered by their *second half*. A **stable** counting sort of that list by first-half class finishes the pair sort. So each round is a couple of linear passes, and the total is O(n log n). In practice we also stop as soon as the number of classes reaches N, which makes random strings finish in very few rounds.

Linear-time constructions exist (DC3/skew by Kärkkäinen–Sanders; SA-IS by Nong–Zhang–Chan). They are worth knowing exist, but the doubling version handles n = $5 \cdot 10^5$ in about 0.1 s and is much harder to get wrong.

3. LCP array: Kasai's algorithm

Define `lcp[i] = LCP(s[sa[i]..], s[sa[i+1]..])` for `0 ≤ i < n−1`. Computing each value naively is O(n) each, O(n^2) total. Kasai et al. (2001) observed:

Lemma. Let `h(i)` be the LCP of suffix i with the suffix that follows it in sa. If `h(i) = k > 0`, then `h(i+1) ≥ k − 1`.

Proof. Let j be the successor of i in sa; suffixes i and j share k characters and s[i..] < s[j..]. Remove the first character from both: suffixes i+1 and j+1 share k−1 characters and still s[i+1..] < s[j+1..]. The successor of i+1 in sa lies between them (or is j+1), and any suffix sorted between two strings that share a prefix of length k−1 also has that prefix. ∎

So process suffixes in text order i = 0, 1, …, n−1 and start the character comparison at `k − 1` instead of 0. The value k increases by at most n in total (it never exceeds n) and decreases by at most 1 per step, so the total work is O(n).

LCP of arbitrary suffixes. For ranks a < b, `LCP(s[sa[a]..], s[sa[b]..]) = min(lcp[a..b−1])`. Build a sparse table over lcp (Year week 24) and each query is O(1). This turns "compare two substrings" into O(1) deterministically — a collision-free replacement for hashing.

4. Applications of SA + LCP

Distinct substrings. Every substring is a prefix of exactly one suffix. Walking sa in order, suffix sa[i] has `n − sa[i]` prefixes, of which the first `lcp[i−1]` already appeared as prefixes of the previous suffix. Hence

#distinct = n(n+1)/2 − Σ lcp[i].

(CSES 2105.) The same scan gives the **k-th smallest distinct substring** (CSES 2108): keep a running count of new prefixes and stop when it reaches k.

Longest repeated substring. `max lcp` (CSES 2106). Longest substring occurring at least k times: the maximum, over windows of k−1 consecutive lcp values, of the window minimum — a sliding-window minimum (monotonic deque, Year week 12).

Pattern matching. Binary search the block of suffixes starting with p: O(m log n) per pattern, and the occurrence count is the block size. With LCP-aware binary search this becomes O(m + log n), rarely necessary.

Longest common substring of s and t. Build the SA of `s + '#' + t` (with `#` not in the alphabet). The answer is the maximum lcp[i] over adjacent pairs whose suffixes come from different strings. Proof: the optimal pair of suffixes (one from each side) can be replaced by an adjacent pair from different sides with at least the same LCP, because LCP over a rank range is a range minimum. For k strings, use a two-pointer window over sa that covers all k colours and take the window's lcp minimum.

LCP intervals and the virtual suffix tree. For a threshold L, maximal sa-ranges where all internal lcp values are ≥ L correspond to substrings of length L; the range size is the number of occurrences. As L varies these ranges nest, forming exactly the internal nodes of the suffix tree. You can enumerate all of them in O(n) with a monotonic stack over lcp: when you pop an interval with lcp value d whose parent has lcp value d′, it represents d − d′ distinct substrings, each with `size` occurrences.

That is the SA solution to the motivating problem **CF 123D *String***: Σ over distinct substrings of occ(occ+1)/2. Each popped interval adds `(d − d′) · size(size+1)/2`, and each leaf (single suffix) adds `(n − sa[i] − max(lcp[i−1], lcp[i]))·1`.

5. Suffix automaton

5.1 endpos classes

For a non-empty substring t of s, let `endpos(t)` be the set of positions where occurrences of t end. Three facts (all short proofs, see the further reading):

1. If u is a suffix of w, then `endpos(w) ⊆ endpos(u)`; otherwise the two sets are disjoint.
2. Substrings with the same endpos set are suffixes of one another, and their lengths form a contiguous interval `[minlen, len]`.
3. If v is a class with longest string w, then cutting characters off the front of w keeps you in v until some point where endpos strictly grows; that next class is the **suffix link** `link(v)`, and `minlen(v) = len(link(v)) + 1`.

Because of (1), endpos sets form a laminar family, so the suffix links form a **tree** rooted at the class of the empty string. The automaton's states are the classes, its transitions are "append a character", and it is the minimal DFA accepting exactly the substrings of s (well, suffixes if you mark terminals).

Size bounds. At most $2n - 1$ states (each extension adds at most two states; the first two add one each) and at most $3n - 4$ transitions for $n \geq 3$. The number of distinct substrings is `Σ_v (len(v) − len(link(v)))`.

5.2 Online construction

We append characters one at a time. Let `last` be the state of the whole current string. Appending c:

1. Create `cur` with `len = len(last) + 1`.
2. Walk `p` from `last` along suffix links, adding transition `p −c→ cur` while p has no c-transition. Those are exactly the suffixes of the old string that, followed by c, were never seen before.
3. If we fall off the root (`p = −1`), every suffix of the new string is new: `link(cur) = 0`.
4. Otherwise let `q = next(p, c)`. If `len(p) + 1 = len(q)` the edge is **solid**: q's longest string is exactly (longest of p)+c, so `link(cur) = q`.
5. Otherwise q contains strings longer than len(p)+1 whose endpos did *not* gain the new position. Split q: create `clone` with `len = len(p)+1`, copying q's transitions and link; redirect `p −c→ q` edges to clone while walking further up p's suffix links; set `link(q) = link(cur) = clone`.

Complexity. Steps 2 and 5 are loops. The loop in step 2 adds transitions, of which there are O(n) in total. For step 5, the standard potential argument tracks the depth of `link(last)` in the link tree: each redirected edge decreases that depth, and it rises by O(1) per extension, so the total is O(n). With `array<int,26>` per state the build is O(nσ) memory; with `std::map` it is O(n log σ) time.

5.3 Applications

Occurrence count. `|endpos(v)|` equals the number of prefixes of s whose state lies in v's subtree of the link tree. So give weight 1 to every state reached by a prefix (all non-clone states) and sum weights bottom-up. Processing states in decreasing `len` is a valid bottom-up order since `len(link(v)) < len(v)`; counting sort makes this O(n).

- CF 123D: answer = Σ_v (len(v) − len(link(v))) · occ(v)(occ(v)+1)/2.
- CF 802I *Fake News (hard)*: Σ_v (len(v) − len(link(v))) · occ(v)².
- CSES 2110 *Substring Distribution*: difference array over lengths `len(link(v))+1 .. len(v)` (just each state's length interval — no occ needed).

Longest common substring. Build SAM(s) and stream t through it maintaining (state, matched length). On a missing transition, follow links (shortening the match to `len(link)`) until the transition exists or we reach the root. This is the "Aho–Corasick failure link" idea, and it is O(|t|) amortized (SPOJ LCS). For many strings (SPOJ LCS2), for every state keep, per string, the best match length ending there, propagate maxima to ancestors (capped at `len(ancestor)`), and take the minimum across strings.

k-th lexicographic substring. `paths(v)` = number of non-empty paths from v in the DAG; compute in reverse topological order (decreasing len). Then greedily descend from the root in letter order, subtracting whole subtrees you skip. With repetitions counted (CF 128B, CSES 2109), each step into state u contributes `occ(u)` instead of 1.

Substring in a range. "Does t occur in s[l..r]?" requires the actual endpos set. Build a segment tree of endpos for each state by **merging** children's trees bottom-up on the link tree (mergeable segment trees, or small-to-large sets — Lecture 4). This is the core of CF 1037H *Security* and CF 666E *Forensic Examination*. The link tree of SAM(s) is the suffix tree of reverse(s), which explains why these merges look like suffix-tree DFS.

Generalized SAM. For several strings, either insert them into one automaton resetting `last = 0` before each string (with the extra check that the target state might already exist), or build a trie and BFS-insert. The simple "separator" trick (`s1#s2$…`) also works when you only need substrings without separators.

6. Reference implementations

Both programs compile with `g++ -std=c++17 -O2 -Wall` and were stress-tested against brute force on thousands of random small strings.

6.1 Suffix array, LCP, distinct substrings

Input: one string. Output: sa, lcp, number of distinct substrings.

```cpp
#include <bits/stdc++.h>
using namespace std;

// Suffix array in O(n log n): sort cyclic shifts of s + '\0'.
vector<int> suffixArray(const string& str) {
    string s = str + char(0);           // sentinel smaller than every char
    int n = s.size(), A = 256;
    vector<int> p(n), c(n), pn(n), cn(n), cnt(max(A, n), 0);
    for (char ch : s) cnt[(unsigned char)ch]++;
    for (int i = 1; i < A; i++) cnt[i] += cnt[i - 1];
    for (int i = n - 1; i >= 0; i--) p[--cnt[(unsigned char)s[i]]] = i;
    int classes = 1; c[p[0]] = 0;
    for (int i = 1; i < n; i++) {
        if (s[p[i]] != s[p[i - 1]]) classes++;
        c[p[i]] = classes - 1;
    }
    for (int h = 1; h < n && classes < n; h <<= 1) {
        for (int i = 0; i < n; i++) pn[i] = (p[i] - h + n) % n;   // sort by 2nd half:
shift
        fill(cnt.begin(), cnt.begin() + classes, 0);
        for (int i = 0; i < n; i++) cnt[c[pn[i]]]++;
        for (int i = 1; i < classes; i++) cnt[i] += cnt[i - 1];
        for (int i = n - 1; i >= 0; i--) p[--cnt[c[pn[i]]]] = pn[i];  // stable by 1st
half
        cn[p[0]] = 0; classes = 1;
        for (int i = 1; i < n; i++) {
            pair<int,int> cur = {c[p[i]], c[(p[i] + h) % n]};
            pair<int,int> prv = {c[p[i - 1]], c[(p[i - 1] + h) % n]};
            if (cur != prv) classes++;
            cn[p[i]] = classes - 1;
        }
        c.swap(cn);
    }
    p.erase(p.begin());                 // drop the sentinel suffix
    return p;
}

// Kasai: lcp[i] = LCP(suffix sa[i], suffix sa[i+1]), size n-1.
vector<int> lcpArray(const string& s, const vector<int>& sa) {
    int n = s.size();
    vector<int> rk(n), lcp(max(n - 1, 0));
    for (int i = 0; i < n; i++) rk[sa[i]] = i;
    for (int i = 0, k = 0; i < n; i++) {
        if (k > 0) k--;
        if (rk[i] == n - 1) { k = 0; continue; }
        int j = sa[rk[i] + 1];
        while (i + k < n && j + k < n && s[i + k] == s[j + k]) k++;
        lcp[rk[i]] = k;
    }
    return lcp;
}

int main() {
    ios::sync_with_stdio(false); cin.tie(nullptr);
```

```
    string s;
    cin >> s;
    vector<int> sa = suffixArray(s), lcp = lcpArray(s, sa);
    long long n = s.size(), distinct = n * (n + 1) / 2;
    for (int x : lcp) distinct -= x;
    for (int i = 0; i < (int)n; i++) cout << sa[i] << " \n"[i + 1 == n];
    for (int i = 0; i + 1 < (int)n; i++) cout << lcp[i] << " \n"[i + 2 == n];
    cout << distinct << "\n";
}
```

6.2 Suffix automaton with occurrence counts and LCS

Input: strings s and t (lowercase). Output: number of distinct substrings of s, length of the longest common substring of s and t, the maximum occurrence count among substrings of s of length ≥ 2 (0 if none), and the number of states.

```
#include <bits/stdc++.h>
using namespace std;

struct SAM {
    struct State { int len, link; array<int, 26> next; };
    vector<State> st;
    int last;
    SAM(int maxLen = 0) {
        st.reserve(2 * maxLen + 2);
        st.push_back({0, -1, {}});
        st[0].next.fill(-1);
        last = 0;
    }
    int newState(int len) {
        st.push_back({len, -1, {}});
        st.back().next.fill(-1);
        return (int)st.size() - 1;
    }
    void extend(char ch) {
        int c = ch - 'a';
        int cur = newState(st[last].len + 1);
        int p = last;
        while (p != -1 && st[p].next[c] == -1) { st[p].next[c] = cur; p = st[p].link; }
        if (p == -1) st[cur].link = 0;
        else {
            int q = st[p].next[c];
            if (st[p].len + 1 == st[q].len) st[cur].link = q;
            else {
                int clone = newState(st[p].len + 1);
                st[clone].next = st[q].next;
                st[clone].link = st[q].link;
                while (p != -1 && st[p].next[c] == q) { st[p].next[c] = clone; p =
st[p].link; }
                st[q].link = st[cur].link = clone;
            }
        }
```

```
        last = cur;
    }
};

// Longest common substring of s and t using the automaton of s.
int longestCommon(const SAM& a, const string& t) {
    int v = 0, l = 0, best = 0;
    for (char ch : t) {
        int c = ch - 'a';
        while (v != 0 && a.st[v].next[c] == -1) { v = a.st[v].link; l = a.st[v].len; }
        if (a.st[v].next[c] != -1) { v = a.st[v].next[c]; l++; }
        best = max(best, l);
    }
    return best;
}

int main() {
    ios::sync_with_stdio(false); cin.tie(nullptr);
    string s, t;
    cin >> s >> t;
    SAM a(s.size());
    for (char ch : s) a.extend(ch);
    long long distinct = 0;
    for (int v = 1; v < (int)a.st.size(); v++) distinct += a.st[v].len -
a.st[a.st[v].link].len;
    // occurrence counts: endpos sizes via counting sort by len
    int m = a.st.size();
    vector<long long> occ(m, 0);
    {   // each prefix of s ends in a non-clone state; give it weight 1
        int v = 0;
        for (char ch : s) { v = a.st[v].next[ch - 'a']; occ[v] = 1; }
    }
    // counting sort states by len, then push counts up the suffix-link tree
    vector<int> bucket(s.size() + 1, 0), order(m);
    for (int v = 0; v < m; v++) bucket[a.st[v].len]++;
    for (int i = 1; i <= (int)s.size(); i++) bucket[i] += bucket[i - 1];
    for (int v = m - 1; v >= 0; v--) order[--bucket[a.st[v].len]] = v;
    for (int i = m - 1; i > 0; i--) occ[a.st[order[i]].link] += occ[order[i]];
    // most frequent substring of length >= 2 frequency
    long long bestOcc = 0;
    for (int v = 1; v < m; v++) if (a.st[v].len >= 2) bestOcc = max(bestOcc, occ[v]);
    cout << distinct << " " << longestCommon(a, t) << " " << bestOcc << " " << m << "\n";
}
```

Note on `occ[v] = 1` via walking s: the state reached by reading the prefix s[0..i] is the state of that whole prefix, which is the non-clone state created when character i was appended. Walking the prefixes is a convenient way to mark them without storing an `isClone` flag.

7. Common mistakes

- **No sentinel in the doubling SA.** Without `'\0'`, cyclic shifts of `aaa` are all equal and you get an arbitrary order. With a sentinel, also remember to remove its suffix from the result.
- **Signed `char`.** Characters ≥ 128 become negative indices. Cast to `unsigned char`.
- **LCP indexing conventions.** Some references define `lcp[i]` between `sa[i−1]` and `sa[i]`. Pick one convention for your team notebook and write it in a comment.
- **Separator collisions.** When concatenating strings, separators must be distinct from each other and from the alphabet, otherwise an LCP can run across a separator.
- **SAM clone bugs.** The clone copies both `next` and `link`; its `occ` initial weight is 0; and the redirect loop must stop as soon as `next(p,c) ≠ q`.
- **SAM memory.** 2n states × 26 ints is ~208 bytes per character. At n = 10^6 that is over 200 MB. Use `map<char,int>`, a compact edge list, or switch to SA.
- **Reference invalidation.** Taking `State& x = st[v]` and then calling `newState` may reallocate the vector. Reserve `2n` states up front.
- **Overflow.** Counts of substrings are Θ(n^2); use `long long` (and sometimes `__int128` for Σ $occ^2 \cdot length$).

8. Practice problems

Warm-up (direct applications):

- CSES 2105 — Distinct Substrings — https://cses.fi/problemset/task/2105
- CSES 2106 — Repeating Substring — https://cses.fi/problemset/task/2106
- CSES 2108 — Substring Order I — https://cses.fi/problemset/task/2108
- CSES 2109 — Substring Order II — https://cses.fi/problemset/task/2109
- CSES 2110 — Substring Distribution — https://cses.fi/problemset/task/2110
- Library Checker — Suffix Array — https://judge.yosupo.jp/problem/suffixarray
- Library Checker — Number of Substrings — https://judge.yosupo.jp/problem/number_of_substrings

National level:

- SPOJ LCS — Longest Common Substring — https://www.spoj.com/problems/LCS/
- SPOJ LCS2 — Longest Common Substring II — https://www.spoj.com/problems/LCS2/
- Codeforces 123D — String — https://codeforces.com/problemset/problem/123/D
- Codeforces 128B — String — https://codeforces.com/problemset/problem/128/B
- Codeforces 802I — Fake News (hard) — https://codeforces.com/problemset/problem/802/I
- Codeforces 316G3 — Good Substrings — https://codeforces.com/problemset/problem/316/G3
- Codeforces 427D — Match & Catch — https://codeforces.com/problemset/problem/427/D
- Codeforces 873F — Forbidden Indices — https://codeforces.com/problemset/problem/873/F
- Codeforces 235C — Cyclical Quest — https://codeforces.com/problemset/problem/235/C

Stretch (SAM + mergeable structures):

- Codeforces 1037H — Security — https://codeforces.com/problemset/problem/1037/H
- Codeforces 666E — Forensic Examination — https://codeforces.com/problemset/problem/666/E

Suggested order: do each CSES problem twice, once with SA and once with SAM. Then 123D both ways — comparing the LCP-interval solution with the link-tree solution is the best way to internalize that they describe the same tree.

9. Further reading

- cp-algorithms: "Suffix Array" and "Suffix Automaton" — https://cp-algorithms.com/string/suffix-array.html, https://cp-algorithms.com/string/suffix-automaton.html (proofs of the $2n-1$ / $3n-4$ bounds and of linear construction).
- U. Manber, G. Myers, "Suffix arrays: a new method for on-line string searches", SIAM J. Computing, 1993.
- T. Kasai, G. Lee, H. Arimura, S. Arikawa, K. Park, "Linear-time longest-common-prefix computation in suffix arrays and its applications", CPM 2001.
- A. Blumer et al., "The smallest automaton recognizing the subwords of a text", Theoretical Computer Science, 1985.
- G. Nong, S. Zhang, W. H. Chan, "Linear suffix array construction by almost pure induced-sorting" (SA-IS), DCC 2009.
- Next in this track: Lecture 7 (advanced math). In the World Finals track, Lecture 8 extends this material with the palindromic tree, suffix trees, and Lyndon factorization.